

CryptoPulse AI — Complete Project Documentation & Master Guide

Full-Stack AI-Powered Cryptocurrency Tracking, Machine Learning Price Forecasting & Paper Trading Platform

Project Name: CryptoPulse AI	Architecture: Pure Python Full-Stack (FastAPI + SQLite + ML)
Author / Developer: Sourabh Sheoran	Live Server Port: http://localhost:8000
Core ML Models: HistGradientBoosting, Random Forest	Repository: github.com/Sourabh-sheoran/CryptoApp

1. Executive Summary & Core Purpose

CryptoPulse AI is a production-grade, full-stack cryptocurrency market analytics and algorithmic forecasting platform developed completely in Python. The platform addresses the volatility and complexity of the digital asset markets by providing traders and investors with three fundamental pillars:

- **Real-Time Market Ingestion:** High-frequency live price streaming for top 100+ cryptocurrencies, global market metrics (Total Market Cap, 24h Volume, BTC Dominance), and the Alternative.me Fear & Greed Sentiment Index.
- **Machine Learning Price Prediction Engine:** Automated feature engineering over historical OHLCV data that calculates 27+ statistical/technical indicators (RSI, MACD, Bollinger Bands, Moving Averages) to train Gradient Boosted Regressors predicting next-day Open, High, Low, Close prices with multi-day trajectory forecasting and automated Buy/Sell/Hold signals.
- **Paper Trading Simulator:** Zero-risk virtual portfolio execution with \$50,000.00 USD initial simulated capital, atomic database transactions, real-time P&L; tracking, weighted average cost accounting, and watchlist persistence.

2. Complete Technology Stack & Specifications

Layer / Category	Technologies & Libraries	Purpose & Architectural Rationale
Backend Framework	FastAPI, Uvicorn, ASGI	High-performance asynchronous Python REST framework with automatic OpenAPI/Swagger docs and async I/O.
Database	SQLite 3, Thread-Safe Context	Embedded, zero-configuration relational database with foreign key cascades and atomic ACID transactions.
Security & Auth	Bcrypt, Python-Jose (JWT)	Salted password hashing preventing rainbow attacks, stateless HMAC-SHA256 bearer tokens with 7-day expiration.
Machine Learning	Scikit-Learn, NumPy, Pandas	HistGradientBoostingRegressor & RandomForest ensembles, TimeSeries split, RMSE / R ² evaluation metrics.
Financial Data	yfinance, Httpx, CoinGecko REST	2-year daily historical OHLCV data ingestion, live market tickers, and in-memory TTL caching (45s).
Frontend UI	HTML5, Vanilla JS, CSS3, Chart.js	Cyberpunk dark glassmorphism dashboard, dynamic canvas charting with confidence bands, and responsive modals.

3. Machine Learning Engine & Mathematical Foundations

The Machine Learning subsystem in `app/ml_engine.py` is engineered to perform multi-target regression on sequential financial data without lookahead bias:

A. Mathematical Formulations of Extracted Features:

- Relative Strength Index (RSI - 14 Periods):** Measures directional price momentum on a 0-100 scale:
$$RS = (Avg\ Gain\ over\ 14d) / (Avg\ Loss\ over\ 14d)$$
$$RSI = 100 - (100 / (1 + RS))$$

Interpretation: RSI < 30 indicates oversold conditions (bullish reversal); RSI > 70 indicates overbought conditions (bearish pullback).
- Moving Average Convergence Divergence (MACD):** Captures trend velocity and crossovers:
$$MACD\ Line = EMA(12) - EMA(26)$$
$$Signal\ Line = EMA(9, MACD\ Line)$$
$$Histogram = MACD\ Line - Signal\ Line$$

Interpretation: MACD crossing above Signal line generates a bullish momentum trigger.
- Bollinger Bands (20 Periods, 2 Std Deviations):** Measures dynamic price envelope and volatility:
$$Upper\ Band = SMA(20) + 2 * \sigma(20)$$
$$Lower\ Band = SMA(20) - 2 * \sigma(20)$$
$$Band\ Width = (Upper\ Band - Lower\ Band) / SMA(20)$$
- Autoregressive Lags & Price Range:** Previous 1-day, 2-day, and 3-day Open, High, Low, Close, Volume lags, and Normalized Daily Range $(High - Low) / Close$.

B. Model Training & Trajectory Forecasting Architecture:

- **Algorithm:** HistGradientBoostingRegressor (Histogram-based decision trees with early stopping and L2 regularization) trained independently for 4 targets: *Open, High, Low, Close*.
- **Train/Test Protocol:** 85/15 sequential time-series split (shuffle=False) ensuring zero historical data leakage.
- **Multi-day Forecast Trajectory:** Projects day 1 to day 30 forward with a mean-reverting drift decay factor $decay = 1 / (1 + day * 0.04)$ and a volatility expansion band $\pm Price * (0.015 * sqrt(day))$ providing upper and lower 95% confidence intervals.
- **Evaluation Metrics:** Root Mean Squared Error (RMSE) and R² Coefficient of Determination.

4. Relational Database Schema & Data Models

The system utilizes an embedded SQLite database (cryptoapp.db) with strict schema normalization and foreign key constraints:

Table Name	Primary Columns & Types	Key Relationships & Constraints
users	id (INTEGER PK), name (TEXT), email (TEXT UNIQUE), password_hash (TEXT), virtual_balance_usd (REAL DEFAULT 50000.0), created_at	Parent entity for Portfolios, Transactions, and Watchlists.
portfolios	id (PK), user_id (FK), coin_id (TEXT), symbol (TEXT), name (TEXT), quantity (REAL), avg_buy_price_usd (REAL), total_invested_usd (REAL)	UNIQUE(user_id, coin_id). Automatically tracks weighted-average cost basis.
transactions	id (PK), user_id (FK), type ('BUY' 'SELL'), coin_id (TEXT), symbol (TEXT), quantity (REAL), price_usd (REAL), total_usd (REAL), timestamp	Complete immutable trade ledger and execution history.
watchlists	id (PK), user_id (FK), coin_id (TEXT), symbol (TEXT), name (TEXT), added_at	UNIQUE(user_id, coin_id). Stores user starred assets.

5. Complete REST API Specifications

Method & Route	Description	Auth Required?	Key Parameters / Request Body
POST /api/auth/register	Register a new user account with \$50k virtual balance	No	{ name, email, password }
POST /api/auth/login	Authenticate user credentials & issue JWT token	No	{ email, password }
GET /api/auth/me	Retrieve current authenticated user profile & balance	Yes (Bearer)	None
POST /api/auth/reset-balance	Reset paper trading portfolio & restore \$50k funds	Yes (Bearer)	None
GET /api/market/coins	Stream top 100 cryptocurrencies with sparklines	No	currency (usd/inr/eur), limit
GET /api/market/coins/{id}	Get in-depth asset stats, circulating supply, ATH	No	id (e.g. bitcoin), currency
GET /api/market/coins/{id}/historical	Historical price & volume timeseries for Chart.js	No	id, days (1, 7, 30, 90, 365), currency
GET /api/market/fear-and-greed	Live Alternative.me Fear & Greed sentiment index	No	None
GET /api/predict/	Execute ML price forecast & AI trade signals	No	symbol (e.g. BTC-USD), days (1-30)
GET /api/portfolio/	Get user net worth, holdings, live P&L, trade ledger	Yes (Bearer)	None
POST /api/portfolio/buy	Execute simulated Buy order at live market price	Yes (Bearer)	{ coin_id, symbol, name, quantity, price_usd }
POST /api/portfolio/sell	Execute simulated Sell order at live market price	Yes (Bearer)	{ coin_id, symbol, name, quantity, price_usd }
GET/POST/DELETE /api/portfolio/watchlist	View, add, or remove assets from user watchlist	Yes (Bearer)	{ coin_id, symbol, name }

6. Interview Master Cheat Sheet & Ready Responses

Ready-to-Speak 60-Second Elevator Pitch (English):

"I have developed CryptoPulse AI, a full-stack, AI-driven cryptocurrency market analytics and machine learning price forecasting platform built entirely in Python using FastAPI. On the backend, I implemented an embedded SQLite database secured with Bcrypt password hashing and JWT authentication, providing every user with \$50,000 USD in virtual capital for real-time paper trading with live P&L; tracking. The core highlight of the project is its Machine Learning Engine, which fetches historical market data via yfinance and engineers over 27 statistical features including 14-period RSI, MACD, Moving Averages (SMA 20/50), and Bollinger Bands. Using HistGradientBoosting Regressors, the system forecasts future Open, High, Low, and Close prices with confidence interval trajectories and generates automated AI Trading Signals (Buy, Sell, Hold). The frontend is a dark cyberpunk glassmorphism terminal built with Chart.js and Vanilla JS that delivers real-time market data, multi-currency conversion, and Fear & Greed sentiment index."

Ready-to-Speak 60-Second Elevator Pitch (Hinglish):

"Maine CryptoPulse AI develop kiya hai — jo ek Full-Stack AI-Powered Cryptocurrency Tracking, Machine Learning Price Prediction aur Paper Trading Platform hai. Is project ko maine purely Python aur FastAPI par build kiya hai. Backend me maine SQLite database ke sath secure Bcrypt password hashing aur JWT token authentication implement kiya hai, jisme har user ko \$50,000 USD virtual capital milta hai real-time simulated trading ke liye. Iska sabse important component iska Machine Learning Engine hai, jisme hum yfinance se historical OHLCV data fetch karke 27+ statistical features (jaise 14-period RSI, MACD momentum, Bollinger Bands, aur 20/50 SMA) extract karte hain. Phir Histogram Gradient Boosting Regressor model ke through hum future Open, High, Low, Close prices predict karte hain aur multi-day forecast trajectory banate hain with automated AI Trading Signals (Strong Buy, Buy, Hold, Sell). Frontend me maine Chart.js aur Vanilla JavaScript ke sath ek dark cyberpunk trading dashboard banaya hai jo CoinGecko aur Alternative.me se real-time multi-currency data stream karta hai."

Top Technical Interview Questions & Model Answers:

Q1: Why did you choose FastAPI over Flask or Django?

Answer: FastAPI is built on Starlette and Pydantic, making it one of the fastest Python web frameworks available (comparable to Node.js and Go). It natively supports asynchronous concurrency (async/await), automatic request validation, and self-generating OpenAPI/Swagger documentation at /docs.

Q2: How did you prevent data leakage during ML model training on time-series data?

Answer: In financial time-series forecasting, standard random cross-validation leaks future data into past training folds. To avoid this, I used sequential time-series splitting with shuffle=False. Furthermore, all rolling indicators (SMA, RSI, MACD) are strictly computed on historical backward-looking windows.

Q3: Why Gradient Boosted Decision Trees over Deep Learning (LSTM / RNN)?

Answer: For tabular financial data with engineered technical indicators, Gradient Boosted Trees (like HistGradientBoosting) consistently outperform Recurrent Neural Networks while requiring 100x less compute and inference time (< 80ms). They handle non-linear interactions without GPU acceleration.

Q4: How does the Paper Trading module calculate weighted average buy price?

Answer: When a user executes subsequent BUY orders for an existing asset, the average price updates via weighted average formula: $\text{New_Avg} = (\text{Old_Total_Invested} + \text{New_Cost}) / (\text{Old_Quantity} + \text{New_Quantity})$. When selling, cash balance credits with $\text{Proceeds} = \text{Quantity} * \text{Live_Price}$, and active holdings adjust accordingly.